



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

MULTIDISCIPLINARY PROJECT REPORT – MSc IN HPC ENGINEERING

Workload-Driven Exploration of Edge NPU Architectures for Vision-Language-Action Models

subtitle

Author: ALESSANDRO GHIOTTO
Advisor: TOMMASO SPAGNOLO
Academic Year: 2026–2027
Date: September 28, 2026

Abstract

ABSTRACT

1. Introduction

1.1. Problem Statement

1.2. Research Questions

2. Background

Table of contents of this summaries:

- Foundations of DNN accelerators.
 - Efficient Processing of Deep Neural Networks [8] (Sec. 2.1)
 - Eyeriss - An Energy-Efficient Reconfigurable Accelerator for Deep CNNs [5] (Sec. 2.2)
 - TPU — In-Datacenter Performance Analysis of a Tensor Processing Unit [7] (Sec. 2.3)
- Transformer
 - Attention Is All You Need [9] (Sec. 2.4)
 - An Image is Worth 16x16 Words — Vision Transformer [6] (Sec. 2.5)
- Vision-Language-Action models
 - π_0 - A Vision-Language-Action Flow Model for General Robot Control [3] (Sec. 2.6)
 - SmolVLA [4] (Sec. 2.7)
- SotA architectures for VLA
 - Characterizing VLA Models: Identifying the Action Generation Bottleneck for Edge AI Architectures [2](Sec. 2.8)
 - Characterizing Vision-Language-Action Models across Heterogeneous Edge Accelerators [1] (Sec. 2.9)

2.1. Efficient Processing of DNNs

2.1.1 Understanding DNNs

DNN models are different in terms of number of layers, layer types, layer shapes (filter size, #channels, #filters) and connections between layers. It's important to understand all of this variations for incorporating the right flexibility in a DNN accelerator.

- Connections: fully vs sparsely connected, weight sharing, feed forward vs recurrent.
- **Data Types and Tensor Definitions:**
 - **Weights** ($\mathbf{F}$ / f): Model parameters of the filters.
 - **ifmap** ($\mathbf{I}$ / i): Input Feature Map ($C \times H \times W$).
 - **ofmap** ($\mathbf{O}$ / o): Output Feature Map ($M \times P \times Q$).
 - **psum** (Partial Sum): Intermediate accumulation result.
 - **bias** ($\mathbf{b}$ / b): bias (M).
- **Shape Parameters, Index Variables, and Spatial Relations:**
 - N = Batch size ($0 \leq n < N$).
 - M = # filters / output channels ($0 \leq m < M$).
 - C = # input channels ($0 \leq c < C$).
 - H/W = Ifmap spatial height / width.
 - R/S = Filter spatial height / width ($0 \leq r < R, 0 \leq s < S$).
 - P/Q = Ofmap spatial height / width ($0 \leq p < P, 0 \leq q < Q$).
 - U = Stride.
 - Spatial Output: $P = \frac{H-R+U}{U}$, $Q = \frac{W-S+U}{U}$.
- **CONV LAYER:**

$$\mathbf{O}_{nmpq} = \left(\sum_{c=0}^{C-1} \sum_{r=0}^{R-1} \sum_{s=0}^{S-1} \mathbf{I}_{nc(Up+r)(Uq+s)} \mathbf{F}_{mc rs} \right) + \mathbf{b}_m.$$

Where $Up + r$ and $Uq + s$ map the output position (p, q) back to the spatial coordinates of the input feature map window.

- **FC LAYER:** Special case of CONV layer where filter spatial size matches ifmap size ($R = H, S = W$), output spatial resolution is 1×1 ($P = Q = 1$), and stride $U = 1$:

$$\mathbf{O}_{nm} = \left(\sum_{c=0}^{C-1} \sum_{h=0}^{H-1} \sum_{w=0}^{W-1} \mathbf{I}_{nchw} \mathbf{F}_{mchw} \right) + \mathbf{b}_m.$$

Matrix multiplication, memory bound (no weight sharing).

- Non linearities: Sigmoid, tanh, ReLU family.
- Pooling and unpooling: change the spatial resolution.
- Normalization: normalize the input distribution of a layer such that we have zero mean and unit standard deviation.
- Attention layer: matrix multiplication + feed forward + FC layers.

2.1.2 Metrics

To fairly evaluate all the design trade-offs, ALL the following metrics should be accounted.

- **Accuracy:** quality of the results. It should be interpreted in the context of the difficulty of the task.
- **Latency:** the time between when the input data arrives to a system and when the result is generated.
- **Throughput:** #OPs/s. Depends on the complexity of the single operation, the clock frequency, the number of active PEs, the utilizations of the PEs, the number of ineffectual operations exploited by the HW.

Roofline model: throughput is bounded by the bandwidth when the operational intensity (#OPs/byte) is low. Instead it is bounded by the max achievable throughput when the operational intensity is high.

- **Energy efficiency:** amount of data that can be processed for a given unit of energy.
- Power consumption:** amount of energy consumed per unit time.
- Don't only report the energy efficiency and power consumption of the chip, but also the energy efficiency and power consumption of the off-chip memory (DRAM) or the amount of off-chip accesses. The number of MAC operations is not enough, doesn't say how the data is accessed or reused.
- Hardware cost.
- **Flexibility:** the range of DNN models supported.
- **Scalability:** report performance and efficiency metrics as the number of PEs and storage capacity increases.

2.1.3 Designing DNN Accelerators

- **Temporal architecture:** centralized control of many ALUs, which can only fetch data from the main memory, they can't communicate with each other. SIMD/SIMT for improving parallelism.
- **Spatial architecture:** allows for communication between ALUs. PE: each ALU can have its own control logic, and a local memory (scratchpad) or a register file.
- Different MACs can only reuse data for at most one data type at a time.

MAC(ifmap, weights, psums)

- The mapping depends on the HW (fixed) and the model (change). The set of supported mappings is called a *dataflow*.

Optimization(HW design, shape & size of DNN)

- Techniques for exploiting data reuse:
 - **Temporal reuse:** the same data value is used more than once by the PE, this may be exploited by adding an intermediate memory. Data in the intermediate level may be replaced, and it depends on the reuse distance. Decreasing the reuse distance of one data type increases the reuse distance of the others.
 - **Spatial reuse:** the same data value is used by more than one PE at different spatial locations of the HW. Exploiting spatial reuse creates more duplicated data, the other option is to support multicast which is expensive at large scale.
- The order of operations prioritize reducing the reuse distance of one data type over the others, this prioritize data type is stationary over time. stationariness is controlled by the ordering of the nested loops.
- **Tiling.** *Temporal tiling:* maximize the reuse of the values held in the smaller and faster memories. The reuse is higher if a single tile of weights is used repeatedly to create a series of psums, and if the tile is small enough to be held in the memory closest to the compute units. *Spatial tiling:* reuse the same data value by as many PEs as possible, reduce the reuse distance so that one multicast can serve as many PEs as possible given a fixed amount of storage capacity at each PE. The goal of tiling is to **exactly match** the reuse distance of a data type to the available storage capacity at a certain level. Overly reducing the reuse distance doesn't give any benefit, just increase the reuse distance of the other data types.
- **Dataflow:** specifies an ordering, and which operations run in parallel. Described by *loop nests*.
 - Outer loop: stationary values. This data values are loaded only once, they are reused for each inner loop.
 - Inner loop: what I reload each time.
 - Tiling is done by picking a dimension of the corresponding data type and breaking it up into multiple loops.
 - **parallel-for** loops run on parallel PEs

A dataflow describe this aspects of a loop nest: (1) order of the loops (which data types e prioritize); (2) number of loops for each data dimension (describe the tiling); (3) whether each of the loops is temporal or spatial.

- **DATAFLOW TAXONOMY.**
 - **Weight Stationary (WS):** maximize the reuse of weights, while the ifmaps are broadcast and the psums are spatially accumulated.
 - **Output Stationary (OS):** minimize the energy consumption of reading and writing the psums by accumulating them in the PE. While the ifmaps are streamed and the weights are broadcast.
 - **Input Stationary (IS):** minimize the energy consumption of reading the ifmaps, while unique filter weights are uni-cast and the psums are spatially accumulate.
 - **Row Stationary (RS):** maximize the reuse of all types of data for the overall energy efficiency.
- **Pipeline.** It needs: (1) a series of ifmaps needs to be processed; (2) interstage registers; (3) all stages should be balanced.
- **BUFFER MANAGEMENT STRATEGIES.** A dataflow control where and when data is buffered, we need also a *buffering scheme* that should achieve good data orchestration.
 - **Implicit** data orchestration: *caches*. we have a replacement policies, the load request initiator doesn't control the cache hierarchy decisions.
Explicit data orchestration: *scratchpads*, which expose and address range, we have control over the data orchestration (ex: GPU's shared mem). We have instructions for transferring data into and out the scratchpad.
 - **Coupled** staging of data: *load/store*.
Decouple the load request initiator and the response receiver. A DMA engine is responsible for pushing data into FUs' buffers
 - **Explicit Decoupled Data Orchestration (EDDO).** An exemplar are FIFO buffers, but they are not flexible enough. *Buffets* store values in a circular buffer and allows the following operations: push a value; read the head value; read the value at a given offset from the head; read and update a value at a given offset; remove a given number of entries (shrink). Buffets are composable.
- **FLEXIBLE NOC DESIGN.** The NoC should support the data delivery patterns of various dataflows. It should consider: (1) high parallelism; (2) exploit data reuse; (3) can be scaled.

Spatial accumulation $\Rightarrow$ WS
 Temporal accumulation $\Rightarrow$ OS

The varying amount of data reuse for each type across layers or models pose a challenge to the NoC design.

Hierarchical Mesh Network: we have an all-to-all network within the scope of a cluster, then the clusters are further connected with a mesh network. This allows for an high connectivity, but it is also schelable.

2.1.4 Operation Mapping on Specialized HW

Mapping involves the placement and scheduling in sapce and time of every operation. Steps: (1) determine the dataflow; (2) determine the data tile sized; (3) bind the operations to the HW.

- Mapping = selecting and parametrizing loop nests.
- Architecture: dataflow, buffer sizes (processors' RF) and NoC connectivity. Microarchitecture: detailed informations.
- The map space is constructed by considering the problem spec and the dataflow.
- The Perf/energy of a model considers the problem, the dataflow and the implementation
- A configuration is created by an optimizer, by iterating over the mapping and evaluating the perf/energy of them,

2.1.5 Reducing Precision

- Minimize the number of bits while also maintaing accuracy. Reduces the amount of data movement, the amount of storage required and the cost of the MAC HW.
- **Quantization:** maps a set of values (full precision) to a reduced set of values (reduced precision). Quantization levels = number of values; quantization values = actual values.

- Uniform vs non-uniform quantization.
- Computable vs non computable (look-up table) relationship between the quantized values to the quantization values.
- Exponents = range of values; Mantissa = #unique values for each scaling factor; signed/unsigned - negative number are represented in two's complement.
- ex: 8-bit fixed point for inference, 16-bit floating points for training, 32-bit floating points for the accumulation.
- Optimal quantization depends on the distribution of the data. Mized precision: use different precision on different data types.
- **Varying precision:** when the precision vary across different parts of a DNN model, requires HW support. Ex: we can chose to do one multiplication with 8-bits or 2 with 4-bits.

2.1.6 Exploiting Sparsity

- **Sources of sparsity.**
 - Activation sparsity. Due to ReLUs; correlation in the input data (if the difference is zero between valued of the same neighbourhood of pixels.); upconvolutional layers.
 - Weight sparsity. Repeated weights in a filter, this can be exploited offline, we know the weights. Reordering the weights can reduce the number of weight accesses.
 - **Network pruning.** (1) Weight removal: we decide which weights we want to remove by scoprng, grouping and ranking them. (2) Fine-tuning: update the values of the remaining weights. (3) Scheduling: how many weights to prune in each iteration.
- **TENSORS.**
 - There is a conceptual partitioning between the activity on the tensors (dataflow) and their representation.
 - Tree-based tensor representation (*Fiber-tree* representation).
 - In practice the order of the ranks matters, since it can affect the cost of access.
 - The axis of the matrix are called *ranks*, the individual elements of a rank are called *coordinated*.
 - At each coordinate of a rank we have a value (*payload*), this value is a *fiber* for intermediate ranks, and it's just a value for the lowest ranks. A giber is a set of coordinates and their associated payloads.
 - Implicit: the position serve as coordinate, fibers are 1-D arrays. Explicit: we have also the position, compressed representation of fibers as a coordinate/payload list.
 - Uncompressed or Compressed. Run-lenght encoding, binary mask. . .
- **SPARSE DATAFLOWS.**
 - Supporting sparsity means that we need additional HW to identify non-zero values.
 - A sparse dataflow use traversals of the sparse tensor abstraction.
 - We iterate over the non-zero values only when we travers a sparse tensor, not over array (there we travers all values).
 - A sparse architecture is made of: (1) a dataflow that operates on an abstract representation of a sparse tensor; (2) implementation choices for the concrete data formats for the tensors.

2.1.7 Designing Efficient DNN Models

To achieve efficiency we may also want to design an efficient DNN network architecture. Remark: reducing the number of weights, activations and MACs doesn't necessarily translate to reduced energy consumption. We must also consider the utilization and the cost of data movement, which depends on the HW, we must consider how the DNN model is mapped on the HW!

- We can replace a big filter (5x5) with small filters (a 5x1 + a 1x5).
- The number of input channels can be reduced via 1x1 CONV layers.

- Grouped convolutions: we have G groups, each group has C/G filters, which are "convoluted" with C/G ifmaps. We don't apply all the filters to all ifmaps.
- We can reduce the number of output channels to be generated by reusing old output channels.
- **Neural Architecture Search (NAS)**. 3 components: search space, optimization algorithm, and performance evaluation.
- Knowledge distillation: we train a smaller model (student) to mimic the output of an already trained model (teacher).

2.1.8 Advanced Technologies

- **Processing near memory**. Embedding high density memories on chip. Stacked memory (3D-memory, HBM) where we stack DRAM on top of the chip.
- **Processing in memory**. ifmaps and weights are read out from their respective memories and processed by MAC arrays. We can use also Non-Volatile Memories (NVM), they allows to increase density. The problem is that they have an high cost of programming the cells, so we should have NVMs big enough to hold all the weights. A drawback of IMC is that the utilization drops if we can't fill entire rows or columns.

2.2. Eyeriss

2.2.1 System Architecture

- **Spatial Architecture**: 168 PEs organized in a 12×14 array operating in a core clock domain.
- Each PE can either communicate with its neighbor PEs or the Global Buffer (GLB) through a NoC, or acces a local memory (spad). There are four levels of memory hierarchy: DRAM, GLB, inter-PE communication, and spads.
- The PEs run independently of each other, they don't proceed in lock steps, so they don't behave as a systolic array.
- **Row Stationary (RS) dataflow**.
 - minimizes data movement for all data types to achieve the best energy efficiency.
 - *1-D Convolutional Primitive*: each primitive operaates on one row of filter weights and one row of ifmaps values, and generate one row of psums.
 - *2-D Convolution PE set*: it's composed of many 1-D convolutional primitives, and it's computation: (1) shares the same row of filter of ifmap across primitives, (2) accumulates the psums from multiple primiteves together.
 - If the spad size is large enough, each PE can run multiple 1-D CONV simultaneously by interleaving their computation.
 - The PE array can fit more than on PE set if the set is small enough. This increase utilization, reuse of the ifmaps, further accumulation of the psums.
 - The GLB buffers ifmpas and psums. The ifmaps can be reused across multiple processing passes; the psums use the GLB as the intermediate storage.
- **Run-Length Compression (RLC)**. By using a compression scheme we can reduce the number of DRAM accesses and skip unnecessary computations. RLC is used to exploit the zeros in the fmaps (outputs of ReLUs). The number of consecutive zeros are represented using 5-b (*run*), the next value is inserted as a 16-b *level*. In 64-b we have: [run, level, run, level, run, level, 0].

2.2.2 System Modules

- *Global Buffer*. It communicates with DRAM through the asynchronous interface and with the PE array throught the NoC. It is divided into 25 reconfigurable SRAM banks (512-bit $\times$ 64-bit, 4 kB each) dynamically allocated between ifmaps and psums.
- *NoC*. It manages data delivery between the GLB and the PE array, and the data delivery between different PEs. Requirements: it should support the data delivery patterns used in the RS dataflow,

should leverage the data reuse achieved by the RS dataflow, and should provide enough bandwidth. It has 3 different parts:

- *Global Input Network (GIN)*: Two-level hierarchy (1 vertical Y-bus $\rightarrow$ 12 horizontal X-buses) delivering single-cycle multicast from GLB to PEs. The same filter weights, ifmaps, psums are delivered to all PEs (multicast).
- *Global Output Network (GON)*: used for reading the psums generated by a processing pass from a PE array back to the GLB. Same architecture as the GIN.
- *Local Network*: between every pair of PEs that are on two consecutive rows of the same column.
- *PE and Data Gating*. The datapath is pipelined into three stages: one for spad access and two for computation. The computation consists of a 16-b two-stage pipelined multiplier and adder. Data gating logic is implemented, we check if the ifmaps are zero to save power.

2.3. TPUs

2.3.1 TPU Architecture

- The TPU was designed as a coprocessor, that you can plug into existing servers like a GPU; but unlike a GPU, the host server sends TPU instructions rather than fetching them itself. The goal was to run whole inference models in the TPU, to reduce interaction with the host CPU, and to be flexible enough to match future DNN needs. The target are inference apps, which favors latency over throughput. Designed for dense matrixes.
- **Blocks:**
 - *Matrix Multiply Unit (MMU)*. It contains 256x256 MACs that can perform 8-b multiply-and-adds on integers.
 - *Accumulators*. The MMU produces 16-bit psums, which accumulate into the 32-bit Accumulators (4 MiB total).
 - Weights are staged through an on chip *Weight FIFO*, that reads from an off-chip DRAM called *Weight Memory*.
 - *Activation*. Applies nonlinear functions.
 - *Unified Buffer (UB)*. It's an on-chip memory holding intermediate results, it can serve as input to the MMU.
 - *DMA Controller*. It transfers the data between the CPU host memory and the UB.
- **CISC Instructions:**
 - *Read_Host_Memory*: CPU mem $\rightarrow$ UB.
 - *Read_Weights*: Weight Memory $\rightarrow$ Weight FIFO.
 - *MatrixMultiply/Convolve*: makes the MMU perform a matrix multiply/convolution from the UB to the Accumulators.
 - *Activate*: performs a nonlinear function, from Accumulators to UB.
 - *Write_Host_Memory*: UB $\rightarrow$ CPU mem.

The aim is to keep the MMU busy. The TPU uses a 4-stage pipeline, where each instruction executes in a separate stage. The aim is to hide the execution of the other instructions by overlapping their execution with the *MatrixMultiply* instruction. CISC instructions can occupy a stage for thousands of clock cycles.

- Since arithmetic is cheap, the MMU uses systolic execution to save energy by reducing reads/writes of the UB. It relies on data from different directions arriving at cells in an array at regular intervals where they are combined.

2.3.2 Performance

- **Roofline Model**. Applications don't fit in on-chip caches, so they are either computation-limited or memory bandwidth-limited. With low operational intensity (FLOPS/Byte) a program is memory bandwidth-limited; with high operational intensity a program is bounded by the max achievable computation rate (FLOPS/sec). The gap between the actual FLOP/sec and the ceiling shows the potential benefit. RESULTS:

- LSTM and MLP are memory-bound, instead CNN is compute-bound.
- In the TPU almost all applications touch the ceiling, for the CPU and the GPU they don't. The reason is the response time, small increases in response time cause customers to use a service less; so the CPU and GPU can't exploit their potentially higher throughput if they don't meet the response-time limit.
- The TPU has no sophisticated feature that consume energy (caches, branch prediction, out-of-order execution...). "Minimalism is a virtue of domain-specific processors".
- The inflection point (where we go from memory-bound to compute-bound) of the TPU it's more on the right wrt the CPU's and GPU's inflection point. Since the TPU has lower bandwidth and higher peak performance.
- **Energy Proportionality:** servers should consume power proportional to the amount of work performed. The TPU has the lowest power, but it has poor energy proportionality: at 10% load, the TPU uses 88% of the power it uses at 100%.
- **Summary TPU success.** Low utilization of a huge, low-cost compute engine still yields high cost-effective performance. The large MMU; the SW-controlled on-chip memory; the ability to run whole inference models to reduce dependence on CPU; a single-threaded, deterministic execution model (proved to be good on 99-percentile response limits); enough flexibility for DNNs; omission of general purpose features (so the TPU can be low power despite the large datapath and memory); the use of 8-b integers.

2.4. Attention is All You Need

2.4.1 The Transformer Architecture

The Transformer is a sequence transduction model (*seq2seq*) relying entirely on self-attention mechanisms, dispensing with recurrent and convolutional layers entirely. It adopts an auto-regressive encoder-decoder structure.

- **Architecture Components:**
 - **Encoder:** Stack of $N = 6$ identical layers. Each layer contains two sub-layers: a multi-head self-attention mechanism and a position-wise fully connected feed-forward network. Residual connections and Layer Normalization are applied around each sub-layer: $\text{LayerNorm}(x + \text{Sublayer}(x))$.
 - **Decoder:** Stack of $N = 6$ identical layers. Inserts a third sub-layer performing multi-head attention over the encoder output stack. Self-attention sub-layers in the decoder are masked to prevent positions from attending to subsequent positions.
- **Names and Shape Parameters:**
 - $N = 6$ (number of identical layers in encoder and decoder stacks),
 - $d_{\text{model}} = 512$ (output dimension of all sub-layers and embedding layers),
 - $h = 8$ (number of parallel attention heads),
 - $d_k = d_v = d_{\text{model}}/h = 64$ (dimension of keys, queries, and values per head),
 - $d_{\text{ff}} = 2048$ (inner-layer hidden dimension of feed-forward networks),
 - n = sequence length,
 - $P_{\text{drop}} = 0.1$ (residual dropout rate).
- **SCALED DOT-PRODUCT ATTENTION:**
Maps queries (Q), keys (K), and values (V) to an output matrix using scaled dot products and softmax normalization:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

The scaling factor $\frac{1}{\sqrt{d_k}}$ counteracts large dot products that would otherwise push the softmax function into regions with extremely small gradients.

- **MULTI-HEAD ATTENTION:**
Linearly projects Q , K , and V h times with learned projections into d_k and d_v dimensions, running attention in parallel across distinct representation subspaces:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O,$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

Projection matrices: $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

– **POSITION-WISE FEED-FORWARD NETWORK (FFN):**

Applied to each position separately and identically across encoder and decoder layers:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

– **POSITIONAL ENCODING:**

Sinusoidal functions added to input embeddings to inject relative and absolute token positioning without recurrence:

$$\text{PE}_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad \text{PE}_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right).$$

2.4.2 Computational Complexity Comparison

Self-attention layers connect all positions with $O(1)$ sequential operations, minimizing path lengths for learning long-range dependencies.

- **Self-Attention:** Per-layer complexity $O(n^2 \cdot d_{\text{model}})$, Sequential Operations $O(1)$, Maximum Path Length $O(1)$.
- **Recurrent Layer:** Per-layer complexity $O(n \cdot d_{\text{model}}^2)$, Sequential Operations $O(n)$, Maximum Path Length $O(n)$.
- **Convolutional Layer:** Per-layer complexity $O(k \cdot n \cdot d_{\text{model}}^2)$, Sequential Operations $O(1)$, Maximum Path Length $O(\log_k(n))$.
- **Restricted Self-Attention:** Per-layer complexity $O(r \cdot n \cdot d_{\text{model}})$, Sequential Operations $O(1)$, Maximum Path Length $O(n/r)$.

2.5. Vision Transformer (ViT)

2.5.1 The ViT Architecture

The Vision Transformer (ViT) applies a standard Transformer encoder directly to 2D image recognition with minimal vision-specific modifications. Images are decomposed into a 1D sequence of flattened, fixed-size 2D patches, allowing visual data to be processed identically to token embeddings in natural language processing.

- **Architecture Components:**
 - **Patch Extraction and Embedding:** An input image is divided into non-overlapping spatial patches, flattened, and linearly projected into a constant latent embedding vector space.
 - **Classification Token:** Prepends a learnable [class] token embedding to the sequence of embedded patches. Its final representation at the output of the encoder serves as the image representation for classification.
 - **Positional Encodings:** 1D learnable position embeddings are added to patch embeddings to preserve spatial topology across sequence tokens.
 - **Transformer Encoder Blocks:** Composed of alternating layers of Multi-Head Self-Attention (MHSA) and MLP blocks (two linear layers with GELU non-linearity). Layer Normalization (LN) is applied before every block, and residual connections are applied after.
- **Names and Shape Parameters:**
 - (H, W) = height and width of the input image,
 - C = number of image channels ($C = 3$ for RGB),
 - (P, P) = spatial resolution of each image patch (e.g., 16×16 or 14×14),
 - $N = \frac{HW}{P^2}$ = number of patches (effective Transformer input sequence length),
 - D = hidden dimension / latent vector size (768 for Base, 1024 for Large, 1280 for Huge),
 - L = number of Transformer encoder layers (12 for Base, 24 for Large, 32 for Huge),
 - h = number of attention heads (12 for Base, 16 for Large, 16 for Huge),

- d_{mlp} = hidden dimension of MLP blocks (3072 for Base, 4096 for Large, 5120 for Huge).
- **PATCH AND POSITION EMBEDDING INJECTION:**

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 C) \times D}, \quad \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}.$$

- **TRANSFORMER ENCODER BLOCK COMPUTATION** ($l = 1 \dots L$):

$$\begin{aligned} \mathbf{z}'_l &= \text{MHSA}(\text{LN}(\mathbf{z}_{l-1})) + \mathbf{z}_{l-1}, \\ \mathbf{z}_l &= \text{MLP}(\text{LN}(\mathbf{z}'_l)) + \mathbf{z}'_l. \end{aligned}$$

- **CLASSIFICATION HEAD:**

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0).$$

Uses an MLP with one hidden layer during pre-training and a single linear layer during fine-tuning.

2.5.2 Inductive Bias vs. Data Scale

- **Lack of Inductive Bias:** Unlike CNNs (which have built-in translation equivariance, locality, and 2D spatial structures across every layer) ViT’s self-attention layers are global. Spatial relations must be learned almost entirely from data.
- **Scale Trumps Inductive Bias:** When trained on mid-sized datasets, ViT yields modest accuracy below CNNs of comparable size. However, when pre-trained on massive datasets, scale overcomes the lack of inductive bias and outperforms traditional CNNs.
- **2D Position Interpolation:** Fine-tuning at higher resolutions keeps patch size P fixed, resulting in larger sequence lengths N . Pre-trained 1D position embeddings are dynamically 2D-interpolated to match the expanded grid.

2.6. π_0 - A VLA Flow Model for General Robot Control

2.6.1 Preliminaries: Conditional Flow Matching

- **Generative flow:**
 - Flow-based generative models learn a transformation from a simple distribution, typically Gaussian noise, to the target data distribution.
 - Instead of directly learning a mapping from noise to data, a continuous flow defines a **velocity field** that specifies how a sample should move at each point in time: $\frac{d\mathbf{x}^\tau}{d\tau} = \mathbf{v}_\theta(\mathbf{x}^\tau, \tau)$.
 - Starting from a sample x^0 drawn from the noise distribution, integrating this velocity field from $\tau = 0$ to $\tau = 1$ produces a sample x^1 from the learned data distribution.
- **Conditional Flow Matching (CFM):**
 - CFM learns the velocity field by explicitly constructing a path between a noise sample ϵ and a target data sample x .
 - A simple linear interpolation is

$$\mathbf{x}^\tau = \tau \mathbf{x} + (1 - \tau) \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

where $\tau = 0$ corresponds to pure noise and $\tau = 1$ to the target sample.

- The model is trained to predict the velocity along this path

$$\mathbf{v}_\theta(\mathbf{x}^\tau, \tau) \approx \frac{d\mathbf{x}^\tau}{d\tau} = \mathbf{x} - \epsilon.$$

- At inference, the target x is unknown. The model starts from random noise x^0 and repeatedly follows the learned velocity field:

$$\mathbf{x}^{\tau+\delta} = \mathbf{x}^\tau + \delta \mathbf{v}_\theta(\mathbf{x}^\tau, \tau).$$

This numerical integration (Forward Euler) gradually transforms the noise into a generated sample.

- **Conditioning:**

- In conditional generation, the velocity field also receives the conditioning information c

$$\mathbf{v}_\theta(\mathbf{x}^\tau, \tau, c).$$

- In π_0 , the generated sample is an action chunk A_t , while the conditioning information is the current robot observation:

$$\mathbf{o}_t = (\text{images, language, robot state}) = [\mathbf{I}_t^1, \dots, \mathbf{I}_t^n, l_t, \mathbf{q}_t].$$

Thus, the learned vector field becomes

$$\mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t),$$

allowing the model to generate actions appropriate for the current visual scene, language instruction, and robot state.

- **Application to robot actions:**

- π_0 generates an entire action chunk

$$\mathbf{A}_t = [a_t, a_{t+1}, \dots, a_{t+H-1}], \quad H = 50,$$

- During training, a noisy action chunk is constructed as

$$\mathbf{A}_t^\tau = \tau \mathbf{A}_t + (1 - \tau) \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

$$\text{data distribution: } p(\mathbf{A}_t | \mathbf{o}_t)$$

$$\mathbf{A}_t^\tau \sim q(\mathbf{A}_t^\tau | \mathbf{A}_t) = \mathcal{N}(\tau \mathbf{A}_t, (1 - \tau) I)$$

and the network learns the corresponding velocity

$$\mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t) \approx \mathbf{u}(\mathbf{A}_t^\tau | \mathbf{A}_t) = \frac{d\mathbf{A}_t^\tau}{d\tau} = \mathbf{A}_t - \epsilon,$$

by optimizing the loss

$$\mathcal{L}^\tau(\theta) = \mathbb{E}_{\substack{\mathbf{A}_t \sim p(\cdot | \mathbf{o}_t), \\ \mathbf{A}_t^\tau \sim q(\cdot | \mathbf{A}_t)}} [||\mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t) - \mathbf{u}(\mathbf{A}_t^\tau | \mathbf{A}_t)||^2].$$

- During inference, π_0 starts from

$$\mathbf{A}_t^0 \sim \mathcal{N}(0, I)$$

and integrates the learned vector field from $\tau = 0$ to $\tau = 1$. The paper uses 10 forward-Euler integration steps, with $\delta = 0.1$:

$$\mathbf{A}_t^{\tau+\delta} = \mathbf{A}_t^\tau + \delta \mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t).$$

2.6.2 Motivation and Overview

- **Goal:** develop a generalist robot policy that can perform many different and dexterous tasks across different robots.
- The key idea is to build a **Vision-Language-Action (VLA)** model on top of a pre-trained **Vision-Language Model (VLM)**. This provides Internet-scale visual and semantic knowledge, which is then combined with robot-specific data and actions.
- π_0 uses **cross-embodiment training**: data from different robots is combined into a single model, including single-arm, dual-arm, and mobile manipulators.
- To represent precise and complex robot actions, π_0 uses **action chunking + flow matching**. Instead of generating one discrete action at a time, the model predicts a chunk of continuous future actions.
- The training follows a **pre-training/post-training** approach:
 - **Pre-training:** use a very large and diverse dataset to learn general physical capabilities.
 - **Post-training:** use smaller, high-quality, task-specific datasets to learn efficient and fluent strategies for particular tasks.

2.6.3 π_0 Architecture

- **Backbone:** π_0 is based on the PaliGemma VLM. Images are encoded and projected into the same embedding space as language tokens. Robot-specific inputs and outputs are then added to the VLM.
- **Inputs:**
 - Multiple RGB images from the robot cameras $\mathbf{I}_t^1, \dots, \mathbf{I}_t^n$.
 - A natural-language command l_t .
 - The robot’s proprioceptive state (joint angles) $\mathbf{q}_t$.
 - A noisy chunk of future actions $\mathbf{A}_t^\tau$.

The aggregated inputs form the observation

$$\mathbf{o}_t = [\mathbf{I}_t^1, \dots, \mathbf{I}_t^n, l_t, \mathbf{q}_t].$$

- **Outputs:** a chunk of $H = 50$ future continuous actions

$$\mathbf{A}_t = [a_t, a_{t+1}, \dots, a_{t+H-1}].$$

- **Action Expert:** a second, smaller set of transformer weights specialized for the robot state and action tokens. It can be viewed similarly to a two-expert mixture:
 - VLM weights process image and language information.
 - Action-expert weights process robotics-specific state and action information.

The whole model could be seen as a decoder-only transformer, where only the output related to the action expert is considered ($\mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t)$).

- **Flow Matching:** instead of predicting discrete action tokens, π_0 models the continuous distribution of possible action chunks. During training, Gaussian noise is added to the true action chunk:

$$\mathbf{A}_t^\tau = \tau \mathbf{A}_t + (1 - \tau) \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

The model learns a vector field that transforms the noisy actions into the target actions.

- At inference, the model starts from random noise and integrates the learned vector field from $\tau = 0$ to $\tau = 1$ ($\delta = 0.1$) with 10 Forward-Euler integration steps.
- The action expert uses **bidirectional attention** between action tokens, allowing all actions in the predicted chunk to interact with each other.
- **blockwise causal attention mask:** Within each block attention is bidirectional, while a block cannot attend to future blocks. The "blocks" are: (1) image + language tokens; (2) robot state tokens; (3) action tokens.
- **Model size:** PaliGemma has 3B parameters and the action expert adds 300M parameters, giving a total of approximately 3.3B parameters. A smaller baseline, π_0 -small, has 470M parameters and does not use VLM pre-training.

2.6.4 Training and Dataset

- The pre-training mixture combines the authors’ dexterous manipulation data with open-source datasets such as OXE, Bridge v2 and DROID.
- The authors’ dataset: 903M timesteps from 7 robot configurations and 68 tasks.
- Different robots have different action and state dimensions. π_0 uses a common maximum dimensionality of 18 and zero-pads robots with smaller action/state spaces. Missing camera inputs are similarly masked.
- The pre-training data is intentionally diverse, even if lower quality. This teaches the model how to deal with different situations and recover from mistakes.
- Post-training uses a smaller, high-quality dataset for a specific task. The amount of data depends on the task.

- For some complex tasks, a separate **"high-level VLM policy"** provides intermediate language commands. For example, instead of simply receiving “bus the table”, the high-level model can produce commands such as “pick up the napkin” or “throw the napkin into the trash”.

2.6.5 Evaluation

- **Out-of-box evaluation:** the pre-trained model is compared with OpenVLA, Octo, and π_0 -small. The full π_0 model and a compute-parity version outperform the evaluated baselines on the reported out-of-box tasks. The comparison highlights two important design choices:
 - **Action chunking + flow matching** allows high-frequency and complex continuous actions.
 - **VLM pre-training** improves the model’s ability to understand language and generalize across tasks.
- **Language following:** π_0 is evaluated with:
 - only the overall task command;
 - intermediate commands supplied by a human;
 - intermediate commands generated autonomously by a high-level VLM.
 The VLM-initialized π_0 shows better language-following performance than π_0 -small.
- **Learning new tasks:** π_0 is fine-tuned on tasks ranging from similar to the pre-training tasks (e.g. stacking bowls) to substantially different tasks (e.g. replacing a paper towel roll or manipulating objects in a drawer). Pre-training generally provides a larger benefit when the downstream task is similar to the pre-training distribution and can provide substantial improvements with limited fine-tuning data.
- **Complex multi-stage tasks:** after fine-tuning, π_0 performs tasks lasting several minutes and requiring many individual behaviors. The hardest tasks benefit particularly strongly from pre-training. Training only on the task-specific data can produce a more limited policy, while pre-training provides a broader set of behaviors and recovery strategies.

2.7. SmolVLA

2.7.1 Model Architecture

A pretrained VLM process the state inputs, and its output features are fed to the action expert which produces continuous actions.

- **VLM.** we have a language decoder which is fed by a sequence of tokens, which are obtained from the input images, instruction, and robot state.
 - **Visual tokens.** They are obtained via a vision encoder. As input we use the global image only, in addition to a pixel shuffle operation (rearranges elements in a tensor of shape $(*, C \times r^2, H, W)$ to a tensor of shape $(*, C, H \times r, W \times r)$, where r is an upscale factor).
 - **Projections.** Linear layers are used to change the dimensions of vectors. (1) project the state to match the VLM dimension; (2) project the actions to match the action expert dimension; (3) project the VLM output features to match the action expert dimension.
 - **Layer skipping.** Only the first $N = 16$ layers of the VLM are used.
- **Flow matching action expert.** Similarly to π_0 , the action expert $\mathbf{v}_\theta$ is trained to predict an action chunk starting from VLM features $\mathbf{A}_t = [a_t, a_{t+1}, \dots, a_{t+n}]$.
 - $\mathbf{o}_t$ VLM features extracted from an observation o_t .
 - Noisy action: $\mathbf{A}_t^\tau = \tau \mathbf{A}_t + (1 - \tau)\epsilon, \epsilon \sim \mathcal{N}(0, I),$.
 - $\mathbf{v}$ is trained to match $\mathbf{u}(\mathbf{A}_t^\tau | \mathbf{A}_t) = \mathbf{A}_t - \epsilon,$
 - The action expert is trained with the objective:

$$\mathcal{L}^\tau(\theta) = \mathbb{E}_{\substack{\mathbf{A}_t \sim p(\cdot | \mathbf{o}_t), \\ \mathbf{A}_t^\tau \sim q(\cdot | \mathbf{A}_t)}} [||\mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t) - \mathbf{u}(\mathbf{A}_t^\tau | \mathbf{A}_t)||^2].$$

The action expert interacts with the VLM via attention.

- (Masked) Self Attention: action tokens attend to each other in a causal fashion (attend only to past tokens).

- Cross Attention: action tokens (queries) attend to the VLM’s keys and values.
- In the action expert this two kinds of attention mechanism are interleaved.

2.7.2 Pretraining data collected by the community

- Scaling robotics datasets is difficult for this reasons: (1) differences across datasets; (2) reliance on teleoperation by human experts for data collection; (3) high heterogeneity of robots.
- Community datasets are very diverse in terms of robots, control schemes, camera perspectives, and tasks. They also reflect real-world complexity.
- **Task annotation with VLM.** A VLM is used to generate task descriptions, since noise in task annotations and ambiguous placeholders were observed.
- **Camera viewpoint normalization.** Manual mapping of each camera to a standardized view type, since a variability in the camera naming convention was observed.

2.7.3 Asynchronous inference

The action expert outputs *action chunks*, sequences $\pi(o_t) = \mathbf{A}_t$ with $\mathbf{A}_t = [a_t, a_{t+1}, \dots, a_{t+n}]$ being a sequence of low-level commands enqueued in an action queue, originating from an environment observation o_t . When do we trigger a new prediction $\pi(o_{t+1})$?

Let $\mathbf{A}_t$ be the queue of the next action chunks, n the number of action chunks generated by the action expert, and g a threshold. Then we compute $\tilde{\mathbf{A}}_{t+1} \leftarrow \pi(o_{t+1})$ when

$$\frac{|\mathbf{A}_t|}{n} < g,$$

and so the new queue will be $\mathbf{A}_{t+1} \leftarrow \text{AGGR}[\mathbf{A}_t, \tilde{\mathbf{A}}_{t+1}]$. Depending on g we have 3 scenarios:

- **Sequential limit** ($g = 0$). The client drains the entire action chunk before processing a new observation. While we need to compute the next action chunk, the robot is incapable of acting.
- **Asynchronous inference** ($g = 0.7$). The client consumes 30% of the queue before triggering inference for a new action queue (trade-off between $g = 0$ or $g = 1$).
- **Compute-intensive limit** ($g = 1$). An observation is sent at every timestep, which is the most expensive option.

2.7.4 Experimental setup and implementation details

- SmolVLA is evaluated on both **simulated** and **real-world** robotic manipulation tasks.
- **Simulation.** Two benchmarks are considered: LIBERO (40 tasks divided into four categories), Meta-World (50 tasks with different difficulty levels).
- **Real-world.** Four datasets are collected using low-cost SO-100 and SO-101 robotic arms. Pick-and-place, stacking, sorting.
- The main metric is the **success rate (SR)**.
- **Robots:** (1) SO-100 / SO-101 low-cost, open-source, 3D-printable robotic arms, 6 DOF; (2) Panda 7-DOF torque-controlled robotic arm (used in the LIBERO simulator); (3) Sawyer A 4-DOF robotic arm (used in the Meta-World simulator).
- The VLM is kept **frozen**; only the action expert is trained.
- The main SmolVLA model contains approximately **450M parameters**, of which approximately 100M belong to the action expert.
- **Baselines:**
 - π_0 (3.3B parameters).
 - **ACT** (80M parameters), a Conditional Variational Autoencoder (CVAE) policy, that directly regresses continuous action chunks.

2.7.5 Main results

- **Simulation.** SmolVLA performs strongly on both LIBERO and Meta-World. Compared with π_0 , it is approximately 40% faster to train and requires approximately $6\times$ less memory.
- **Real-world.** On the SO-100 robot, the 0.45B SmolVLA achieves an average success rate of 78.3% vs 61.7% for π_0 and 48.3% for ACT.
- **Pretraining.** Pretraining on community datasets substantially improves real-world performance:
 - SmolVLA without VLA pretraining and single-task training: 40%.
 - Multi-task training without pretraining: 51.7%.
 - Multi-task training with community-data pretraining: 78.3%.
- **Asynchronous inference.** Synchronous inference ($g = 0$) gets an higher succes rate but asynchronous ($g = 0.7$) gets shorter completion time.
 - Sync: 78.3% SR, 13.75 s task completion time.
 - Async: 73.3% SR, 9.70 s task completion time.
- **Ablation study.**
 - **Interleaving** cross and self attention gives better performance than using only one or the other.
 - **Causal** self-attention in the action expert is better than bidirectional self-attention.
 - Using the first 16 VLM layers gives a good trade-off between computational cost and performance
 - Using anc action expert hidden dimension $0.75d$ provides a trade-off, where $d = \text{VLM hidden dimension}$.
 - **Flow matching** performs better than regression.
 - Providing robot states into the VLM is better than providing them into the action expert.
 - Chunk sizes between 10 and 50 actions provide a good trade-off between responsiveness and inference efficiency.
 - Updating the observation every 1 or 10 actions gives the best performance. Executing many actions before processing a new observation reduces performance (trade-off btw inference efficiency vs responsiveness to changes in the environment).
- Summary SmolVLA: small pretrained VLM; VLM layer skipping; reduced number of visual tokens; compact flow-matching action expert; interleaved cross/self-attention; community-driven pretraining data; asynchronous inference;

2.8. VLA models: Identifying the Action Generation Bottleneck for Edge AI Architectures

2.8.1 Preliminaries: MolmoAct

Unlike π_0 and SmolVLA that directly map multimodal inputs to continuous actions via a vector field, MolmoAct is a **Action Reasoning Model (ARM)**. The inference has three stages:

- **Depth Tokenization.** The Vision-Language Model (VLM) backbone autoregressively outputs discrete spatial depth perception tokens (quantized via VQ-VAE, e.g., 100 tokens per resized 320×320 image frame).
- **Spatial Planning:** Conditioned on the depth tokens, the VLM autoregressively generates 1 to 5 visual waypoint coordinates $(u_t, v_t) \in [0, 255]$ representing target gripper trajectory traces in image space. This is like a "visual Chain-of-thought reasoning", we generate a plan for the action we want to perform.
- **Action Expert:** Conditioned on the VLM's hidden states and KV-cache generated across the previous stages, a decoder-only flow-matching action expert executes N_{step} ODE integration steps to denoise a continuous action chunk $a_{t:t+k} \in \mathbb{R}^{k \times d_a}$.

2.8.2 VLA Models

- **Vision Encoder.** Get features embeddings by processing raw pixel data. The embeddings are projected to match the embedding space of the next component.

- **Reasoning.** A decoder-only transformers takes as input the visual and textual tokens and it may generate intermediate outputs such as "Chain-of-Thought" reasoning or spatial waypoint.
- **Action Transformer.** Generate motor-commands

2.8.3 Methodology

- **Real hardware characterization.** Profile the MolmoAct model on Nvidia Jetson Orin and Thor, by instrumenting the PyTorch runtime using NVIDINSight compute to capture kernel-level execution traces.
- **Performance projection and simulation.** Via an XPU simulator the model is decomposed into stages, each of them is modeled as a multi-layer Transformer backbone (and each layer is decomposed in a sequence of operators). Key features of the simulator:
 - Micro-architectural fidelity: the cost models incorporate specific HW details.
 - Analytical roofline: performance on individual operators is estimated using the roofline model.
 - Cross-operator optimization: the framework performs optimization across operator boundaries to model effective prefetching.

2.8.4 Results

- The generation phase (auto-regressive decode with reasoning) is the primary bottleneck in VLA models ($\approx 75\%$ of the full model latency).
- The generation phase is memory-bound. Thor provides $5\times$ the compute of Orin, but the total latency improves by $1.4\times$.
- They simulated the latency if the model scaled up to 100B parameters with scaling laws, and tested with different memory systems. Improved bandwidth from GDDR7 and Process In Memories improves performances.

2.9. VLA models: Heterogeneous Edge Accelerators

3. Methodology

4. Workload Characterization & Profiling

5. Conclusions

References

- [1] Authors. Characterizing vision-language-action models across heterogeneous edge accelerators. *arXiv preprint arXiv:2604.24447*, 2026.
- [2] Authors. Characterizing vla models: Identifying the action generation bottleneck for edge ai architectures. *arXiv preprint arXiv:2603.02271*, 2026.
- [3] Kevin Black, Karl Pertsch, Joey Hejna, Cristian Bodnar, Quan Vuong, Pranav Sanketi, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [4] Rémi Cadene, Simon Alibert, Quentin Gallouédec, Edward Beeching, Zhaorun Zou, and Thomas Wolf. Smolvla: A compact vision-language-action model for efficient robot manipulation. *arXiv preprint arXiv:2506.01844*, 2025.
- [5] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits*, 52(1):127–138, 2016.
- [6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

- [7] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12, 2017.
- [8] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. *Efficient Processing of Deep Neural Networks*. Synthesis Lectures on Computer Architecture. Springer International Publishing, Cham, 1st edition, 2020.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.